

杭州电子科技大学

深度学习

GAN 模型实验

实 验 报 告

学 院

网络空间安全

专 业

网络空间安全

班 级

Xx

学 号

Xx

学生姓名

xx

教师姓名

张烨菲、苏方方

完成日期

2025.12.15

成 绩

GAN 模型编程实验

一、 实验任务

1. 阅读并运行实验提供的 GAN 生成手写字的完整代码，结合课堂所学内容，详细描述并分析每段代码的编码方式，如数据加载与预处理方式、生成器和判别器的网络结构、损失函数与训练过程、测试函数；
2. 对实验中的关键参数进行调整，观察并分析其对生成结果的影响，如学习率、网络结构、训练函数等，至少完成 2 种不同设置的对比实验，给出生成样本的可视化结果。
3. 将 GAN 改成 DCGAN，分析对生成结果的影响。
4. 实验总结。

二、 实验分析

实验环境：

本次实验在本机上运行，采用一块 NVIDIA GeForce RTX 4060 Laptop GPU，环境中 python 版本为 3.9，cuda 版本为 12.1，下载了 torch、matplotlib 等环境包。

代码运行：

- 1、安装与导入依赖：这段代码是实验的环境初始化环节，作用是导入深度学习任务所需的工具库，并确认运行环境，为后续的 GAN 实现做准备。观察图 1 输出可知当前环境中有可用的 GPU，PyTorch 版本为 2.5.1+cu121，说明 GPU 环境已就绪。

```
[1]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torchvision.utils import save_image
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
print('PyTorch version:', torch.__version__)

Device: cuda
PyTorch version: 2.5.1+cu121
```

图 1 安装与导入依赖

- 2、数据集加载（MNIST）：这段代码实现了 MNIST 手写数字数据集的预处理、加载与批量读取配置，是 GAN 训练前的“数据准备环节”，确保后续训练能获取格式化的输

入数据。

如图 2 所示，`transforms.Compose(...)`将多个数据预处理操作“串联”成一个管道，对数据按顺序执行操作。`transforms.ToTensor()`把 MNIST 数据集原始的“PIL 图像格式（像素值范围 0-255）”转换为 PyTorch 张量，同时将像素值缩放到[0, 1]区间。`transforms.Normalize((0.5,), (0.5,))`对张量数据做归一化处理，公式为： $\text{output} = (\text{input} - \text{均值}) / \text{标准差}$ ，这里均值和标准差均设为 0.5，会将数据缩放到[-1, 1]区间（适配 GAN 中常用的输出激活函数 Tanh 的取值范围）。`datasets.MNIST(...)`则是加载 MNIST 数据集，并将前面定义的预处理管道应用到数据上；`DataLoader(...)`则是构建“批量数据加载器”，`batch_size=64`意味着每次训练时输入模型的样本数为 64 个；`shuffle=True`代表每次迭代前打乱训练集顺序，避免模型学习到数据的“顺序规律”，提升泛化能力。最后能看到 MNIST 训练集总共有 60000 个样本。

▼ 2. 数据集加载 (MNIST)

MNIST 是一个经典的手写数字数据集，包含 0-9 共 10 类。

```
[2]: transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

print('Number of training samples:', len(train_dataset))

Number of training samples: 60000
```

图 2 数据集加载

3、定义生成器 (Generator):这一段代码将随机噪声转换为与真实数据 (MNIST 手写数字) 分布一致的图像张量，通过“全连接层 + 激活函数”的序列结构，实现了噪声到图像的映射。

如图 3 所示，生成器类继承自 `nn.Module`，`noise_dim=100`表示输入的随机噪声维度为 100，`self.net = nn.Sequential(...)`用 `nn.Sequential` 按顺序堆叠网络层，构成生成器的“前向计算管道”：`nn.Linear(noise_dim, 256)`为第 1 层全连接层，将 100 维随机噪声映射为 256 维特征向量。`nn.ReLU(True)`为 ReLU 激活函数 (`inplace=True` 表示直接在原张量上运算，节省内存)，引入非线性，避免模型退化为线性映射。`nn.Linear(256, 512)/nn.Linear(512, 1024)`后续全连接层，逐步扩大特征维度，提取更复杂的特征。`nn.Linear(1024, 28*28)`最后一层全连接层，将 1024 维特征映射为 28*28=784 维向量（对应 MNIST 图像的像素数量）；

`nn.Tanh()` 为输出激活函数，将结果缩放到 $[-1, 1]$ 区间，与之前 MNIST 数据的归一化范围 $[-1, 1]$ 保持一致。前向传播中接入定义好的网络结构，再通过 `out.view(-1, 1, 28, 28)` 将 784 维向量重塑为图像张量形状。

3. 定义生成器 (Generator)

生成器的作用：将随机噪声 z 映射为“看起来像真的”手写数字图像。这里定义了一个全连接网络，结构直观。

```
: class Generator(nn.Module):
    def __init__(self, noise_dim=100):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.ReLU(True),
            nn.Linear(256, 512),
            nn.ReLU(True),
            nn.Linear(512, 1024),
            nn.ReLU(True),
            nn.Linear(1024, 28*28),
            nn.Tanh()
        )

    def forward(self, z):
        out = self.net(z)
        return out.view(-1, 1, 28, 28)

G = Generator()
print(G)

Generator(
  (net): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): ReLU(inplace=True)
    (2): Linear(in_features=256, out_features=512, bias=True)
    (3): ReLU(inplace=True)
    (4): Linear(in_features=512, out_features=1024, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=1024, out_features=784, bias=True)
    (7): Tanh()
  )
)
```

图 3 生成器定义

4、定义判别器 (Discriminator)：这一段代码对输入图像做二分类判断（输出“是真实 MNIST 样本”的概率），与生成器形成“对抗博弈”关系，通过“全连接层 + LeakyReLU 激活 + Sigmoid 输出”的结构，实现图像的真伪判别。

如图 4 所示，同样判别器类也继承自 `nn.Module`，`self.net = nn.Sequential(...)` 按顺序堆叠网络层，构成判别器的前向管道：`nn.Linear(28*28, 512)` 为第 1 层全连接层，输入是展平后的 MNIST 图像（ $28 \times 28 = 784$ 维向量），映射为 512 维特征向量。`nn.LeakyReLU(0.2, inplace=True)` 为 LeakyReLU 激活函数（区别于生成器的 ReLU）：`negative_slope=0.2`：对负输入保留小的梯度（如输入为 -1 时，输出为 -0.2），避免 ReLU 对负输入“置 0”导致的梯度消失问题（适配生成器输出的 $[-1, 1]$ 范围）；`inplace=True`：原地运算，节省内存。`nn.Linear(512, 256)` 为第 2 层全连接层，将 512 维特征压缩为 256 维。

`nn.LeakyReLU(0.2, inplace=True)`再次使用 LeakyReLU，保持非线性与梯度流动。`nn.Linear(256, 1)`为最后一层全连接层，将 256 维特征映射为 1 维输出（对应二分类的“得分”）。`nn.Sigmoid()`则是输出激活函数，将得分缩放到[0,1]区间，代表“输入是真实样本”的概率（0≈生成样本，1≈真实样本）。在前向传播阶段，通过 `x = x.view(x.size(0), -1)`将图像张量展平为向量，然后输入到 `self.net`，输出 1 维的概率值。

▼ 4. 定义判别器 (Discriminator)

判别器的作用：判断输入图像是真实样本还是生成样本。

```
[4]: class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.net(x)

D = Discriminator()
print(D)

Discriminator(
  (net): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Linear(in_features=512, out_features=256, bias=True)
    (3): LeakyReLU(negative_slope=0.2, inplace=True)
    (4): Linear(in_features=256, out_features=1, bias=True)
    (5): Sigmoid()
  )
)
```

图 4 判别器定义

5、定义损失函数与优化器：这段代码定义了 GAN 训练的损失计算规则和模型参数更新工具，是连接“模型输出”与“参数优化”的核心环节。`nn.BCELoss()`即二元交叉熵损失，专门用于二分类任务的损失计算（对应判别器“判断图像真伪”的二分类目标）。具体定义如下：

$$\text{BCELoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (1)$$

其中， y_i 是真实标签（真实图像标签为 1，生成图像标签为 0）； p_i 是判别器输出的“图

像为真实样本”的概率；该损失能有效衡量“判别器预测概率”与“真实标签”之间的差异，是 GAN 对抗训练的核心损失指标。 N 为批量大小。`optim.Adam(...)`选择 Adam 作为优化器，`G.parameters()`指定优化器需要更新的参数，也就是生成器 G 的所有可训练参数。`lr=0.0002`设置学习率（控制参数更新的步长），0.0002 是 GAN 训练中常用的学习率取值。判别器优化器定义与生成器一致，只是输入参数不同。

▼ 5. 定义损失函数与优化器

```
[5]: criterion = nn.BCELoss()

optimizer_G = optim.Adam(G.parameters(), lr=0.0002)
optimizer_D = optim.Adam(D.parameters(), lr=0.0002)
```

图 5 损失函数与优化器定义

6、训练 GAN：这段代码是 GAN 的核心训练逻辑，实现了“判别器与生成器交替训练”的对抗过程，分别完成“判别器区分真假样本”“生成器生成逼真样本”的训练目标。`epochs = 10`定义训练轮数，即整个训练集会被完整遍历 10 次。`noise_dim = 100`与生成器的输入噪声维度保持一致（对应 Generator 类的 `noise_dim` 参数）。

如图 6，这里定义了两层循环，外层 `for epoch in range(epochs)`遍历每一轮训练，内层 `for real_imgs, _ in train_loader`按批次读取训练数据（`real_imgs`是当前批次的真实 MNIST 图像，`_`是图像标签，GAN 训练不需要真实标签，故忽略）。`batch_size = real_imgs.size(0)`获取当前批次的样本数量（适配 `train_loader` 中 `batch_size=64` 的配置）。`real_labels = torch.ones(batch_size, 1)`定义真实样本的标签（值为 1，对应判别器“判断为真实样本”的目标）。`fake_labels = torch.zeros(batch_size, 1)`定义生成样本的标签（值为 0，对应判别器“判断为生成样本”的目标）。

```
epochs = 10
noise_dim = 100

for epoch in range(epochs):
    for real_imgs, _ in train_loader:
        batch_size = real_imgs.size(0)

        real_labels = torch.ones(batch_size, 1)
        fake_labels = torch.zeros(batch_size, 1)
```

图 6 训练前准备

开始训练，首先训练判别器，，如图 7，训练前要先清空历史梯度，接着开始三阶段训练：
判别真实样本：`outputs = D(real_imgs)`用判别器处理真实图像，得到“真实样本为真”的概率。`d_loss_real = criterion(outputs, real_labels)`计算判别器对真实样本的损失（希望 `outputs` 接近 1，故用 `real_labels` 作为目标）。

判别生成样本：`z = torch.randn(...)`生成 100 维的随机噪声（符合标准正态分布），作为生成器的输入。`fake_imgs = G(z)`将生成器将噪声转换为“伪造手写数字图像”。`outputs = D(fake_imgs.detach())`：`detach()`：截断梯度传播——此时仅训练判别器，不需要更新生成器的参数，因此切断生成器到判别器的梯度链路，避免生成器被“误更新”。用判别器处理伪造图像，得到“伪造样本为真”的概率。`d_loss_fake = criterion(outputs, fake_labels)`计算判别器对伪造样本的损失（希望 `outputs` 接近 0，故用 `fake_labels` 作为目标）。

更新判别器参数：`d_loss = d_loss_real + d_loss_fake`为判别器的总损失（需同时“判对真实样本、判对生成样本”）。`d_loss.backward()`则是反向传播计算判别器的参数梯度。`optimizer_D.step()`用优化器更新判别器的参数。

```
# ===== 训练判别器 =====
optimizer_D.zero_grad()
outputs = D(real_imgs)
d_loss_real = criterion(outputs, real_labels)

z = torch.randn(batch_size, noise_dim)
fake_imgs = G(z)
outputs = D(fake_imgs.detach())
d_loss_fake = criterion(outputs, fake_labels)

d_loss = d_loss_real + d_loss_fake
d_loss.backward()
optimizer_D.step()
```

图 7 训练判别器

之后开始训练生成器，如图 8，`optimizer_G.zero_grad()`清空生成器的历史梯度。`outputs = D(fake_imgs)`用判别器处理刚才生成的 `fake_imgs`（无 `detach()`，需让梯度传到生成器）。`g_loss = criterion(outputs, real_labels)`利用损失函数，生成器的目标是“让判别器认为伪造图像是真实的”，因此用 `real_labels`（值为 1）作为目标，计算“判别器误判程度”的

损失。接着更新生成器参数：`g_loss.backward()`反向传播计算生成器的参数梯度。

`optimizer_G.step()`用优化器更新生成器的参数。

```
# ===== 训练生成器 =====
optimizer_G.zero_grad()
outputs = D(fake_imgs)
g_loss = criterion(outputs, real_labels)
g_loss.backward()
optimizer_G.step()
```

图 8 训练生成器

我们打印每一个 epoch 的 loss，观察训练过程，得到如下所示的训练结果：

```
print(f"Epoch [{epoch+1}/{epochs}] D_loss: {d_loss.item():.4f} G_loss: {g_
Epoch [1/10] D_loss: 0.0517 G_loss: 7.2803
Epoch [2/10] D_loss: 1.5910 G_loss: 2.1965
Epoch [3/10] D_loss: 1.3260 G_loss: 1.0129
Epoch [4/10] D_loss: 0.0899 G_loss: 4.0158
Epoch [5/10] D_loss: 0.3031 G_loss: 5.9701
Epoch [6/10] D_loss: 0.1185 G_loss: 5.7266
Epoch [7/10] D_loss: 0.2772 G_loss: 4.6545
Epoch [8/10] D_loss: 0.3428 G_loss: 7.2248
Epoch [9/10] D_loss: 0.2999 G_loss: 5.1283
Epoch [10/10] D_loss: 0.1381 G_loss: 4.1097
```

图 9 训练结果

7、可视化生成结果：这段代码是将生成器输出的“伪造手写数字张量”转换为可视化图像的环节，用于直观验证 GAN 的训练效果。

如图 10，首先，`z = torch.randn(16, noise_dim)` 生成 16 个随机噪声样本（形状为[16, 100]），接着用训练好的生成器 G，将 16 个随机噪声转换为 16 张伪造手写数字图像（张量形状为[16, 1, 28, 28]），`fake_imgs.squeeze()`去掉张量中维度为 1 的轴原张量形状[16, 1, 28, 28]中，1 是通道数（MNIST 为单通道），去掉后形状变为[16, 28, 28]（16 张 28×28 的图像）。`.numpy()`将 PyTorch 张量转换为 NumPy 数组——matplotlib 仅支持 NumPy 格式的数据绘图。`plt.subplots(4, 4)`创建一个“4 行 4 列”的子图网格（共 16 个子图），刚好容纳 16 张伪造图像。`figsize=(4, 4)`设置整个图的尺寸为 4×4 英寸，保证图像显示紧凑清晰。接着 for 循环遍历绘制每一张图像，最后用 `plt.show()`显示图像，结果如图 11 所示。

7. 可视化生成结果

```
7]: z = torch.randn(16, noise_dim)
fake_imgs = G(z).detach()
grid = fake_imgs.squeeze().numpy()
fig, axes = plt.subplots(4, 4, figsize=(4, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(grid[i], cmap='gray')
    ax.axis('off')
plt.show()
```

图 10 可视化结果生成

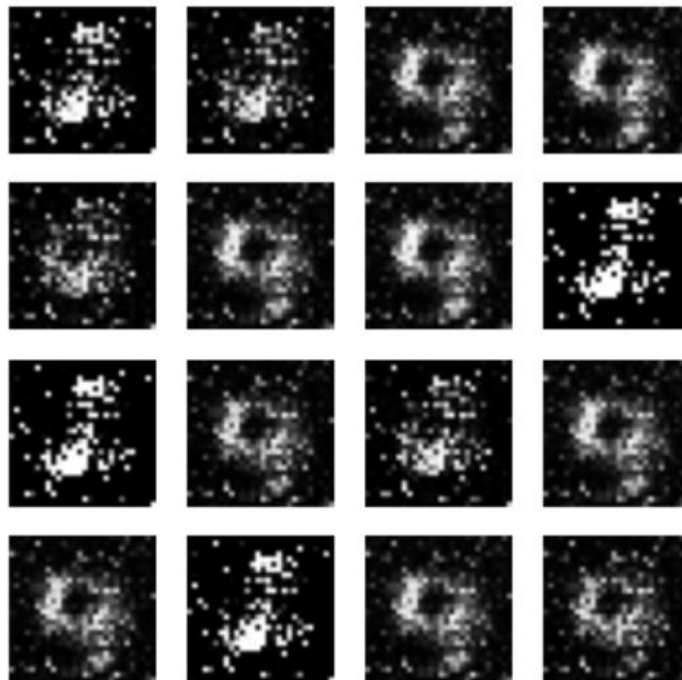


图 11 可视化手写数字结果

实验的一个小分析：

一、损失函数变化分析

从训练过程的 D_loss 和 G_loss 来看：初始阶段（Epoch 1）： $D_loss=0.0517$ （极低）、 $G_loss=7.2803$ （极高）：说明训练初期，判别器能轻松区分“真实样本”和“生成样本”（生成器输出的伪造图像与真实 MNIST 差异极大），生成器的“欺骗能力”极弱。

中间阶段（Epoch 2~10）：损失波动剧烈（如 G_loss 从 Epoch3 的 1.0129 骤升至 Epoch4

的 4.0158，后续持续波动）；最终 Epoch10 的 $D_loss=0.1381$ （仍较低）、 $G_loss=4.1097$ （仍较高）：说明判别器始终处于“优势地位”，生成器的“欺骗能力”未稳定提升，对抗过程未达到平衡状态。

二、可视化结果分析

从生成的 16 张图像来看：图像质量问题：生成的“手写数字”整体模糊、噪点密集，数字轮廓不清晰；分布覆盖问题：图像缺乏“手写数字的典型特征”，未学习到 MNIST 数据集的真实分布。

本次的训练结果 GAN 未达到理想结果，现在尝试一些其他改进。

实验改进：

思考与扩展：

1、如果增大 `noise_dim`，会发生什么？

`noise_dim` 是生成器输入的随机噪声的维度，代表 GAN 的潜在特征空间大小，增大它的影响既有正面效果，也有负面效果：

正面效果：提升生成样本的多样性

①更大的 `noise_dim` 意味着潜在空间的维度更高，能容纳更多的特征组合，生成器可以从更丰富的噪声分布中学习映射关系，理论上能生成更多样化的手写数字（比如不同风格的 0、不同笔画的 9 等）。例如：将 `noise_dim` 从 100 增至 256，潜在空间的自由度更高，生成的数字不会局限于少数几种固定形态。

负面效果：增加训练难度与不稳定性

①训练效率降低：更高维度的噪声需要生成器有更强的特征映射能力（比如需要增加网络层的神经元数量），否则生成器无法有效利用额外的噪声维度，反而会因为输入维度提升导致训练收敛变慢。

②模式崩溃风险增加：如果噪声维度远超过数据的真实特征维度（MNIST 手写数字的核心特征维度其实较低），生成器可能会忽略部分噪声维度，最终只生成少数几种样本（模式崩溃），反而降低多样性。

③计算成本上升：噪声维度增大后，生成器第一层全连接层的参数数量会增加（比如 `nn.Linear(100,256)`变为 `nn.Linear(256,256)`，参数从 25600 增至 65536），训练时的内存和计算量都会上升。

增大 `noise_dim` 应该会存在一个“最优区间”，我们首先尝试将 `noise_dim` 从 100 增至 256 看看实际效果（其余设置均不变）：

```
print('Epoch [{epoch+1}/{epochs}] D_loss: {d_loss:.1f} G_loss: {g_loss:.1f}'
```

Epoch [1/10]	D_loss: 0.0139	G_loss: 7.5119
Epoch [2/10]	D_loss: 0.3796	G_loss: 6.4002
Epoch [3/10]	D_loss: 0.4559	G_loss: 2.3097
Epoch [4/10]	D_loss: 0.2256	G_loss: 4.8772
Epoch [5/10]	D_loss: 0.1539	G_loss: 4.4254
Epoch [6/10]	D_loss: 0.4390	G_loss: 4.3458
Epoch [7/10]	D_loss: 0.1907	G_loss: 5.4378
Epoch [8/10]	D_loss: 0.2444	G_loss: 5.8845
Epoch [9/10]	D_loss: 0.6691	G_loss: 8.7055
Epoch [10/10]	D_loss: 0.4070	G_loss: 3.7047

图 12 训练损失

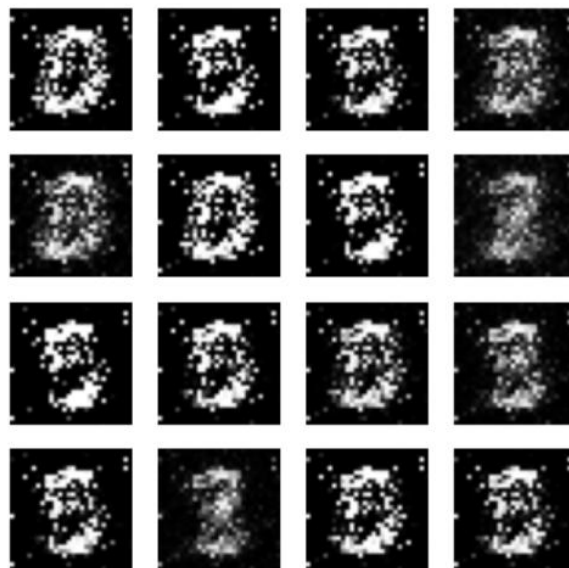


图 13 可视化结果

分析图 12，图 13，较之 `noise_dim=100` 的时候，损失波动更加剧烈，训练稳定性下降，生成图像的白色区域更集中，多样性未提升，反而出现模式单一的趋势。本次训练好像没有明显的提升，甚至存在短板。

为了更好的比较不同参数下的模型性能，我写了一个测试函数，并且采用“**平均像素方差**”

和“**样本间余弦相似度均值**”来衡量生成图像的质量：

平均像素方差是衡量生成的手写数字图像清晰度和纹理特征的显著程度的核心指标，公式如下：

$$Var(p) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W (p_{i,j} - \bar{p})^2 \quad (2)$$

$$\overline{Var(p)} = \frac{1}{N} \sum_{k=1}^N Var(p_k) \quad (3)$$

我们对单张生成图像（ 28×28 的像素矩阵），计算其所有像素值的方差，再对生成的 N 张图像的像素方差取算术平均值，得到平均像素方差，平均像素方差越高越好：

高值：说明生成图像的像素值差异大，有清晰的笔画和背景区分，图像质量高；

低值：说明生成图像的像素值分布均匀，是模糊的噪点，图像质量差。

另一个指标是样本间余弦相似度均值，是衡量生成的手写数字图像多样性的核心指标，反映生成样本之间的差异程度，公式如下：

$$Sim(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \times |\vec{b}|} \quad (4)$$

$$\overline{Sim} = \frac{1}{M} \sum_{(i,j) \in S} Sim(\vec{x}_i, \vec{x}_j) \quad (5)$$

我们将每张生成图像（ 28×28 ）展平为 784 维的一维向量，然后随机选取 200 个样本，计算所有样本对之间的余弦相似度，对所有样本对的余弦相似度取算术平均值，得到最终指标，余弦相似度均值越低越好：

低值：说明生成样本之间的差异大，覆盖的数字类别多，多样性好；

高值：说明生成样本之间的差异小，容易出现“模式崩溃”（只生成少数几种数字），多样性差。

之后我们又测试了不同 noise_dim，从 100，到 256，再到 512，再到 1024，得到如下表：

noise_dim	平均像素方差	样本间余弦相似度均值
100	0.4544	0.8872

256	0.3684	0.9228
512	0.2892	0.8755
1024	0.1808	0.9964

表 1 不同 noise_dim 比较

根据表中数据可知随着 noise_dim 的增加，平均像素方差持续下降，说明生成图像的清晰度、质量持续变差，样本间余弦相似度均值整体上升，说明生成样本的多样性持续降低，总结来说 noise_dim 在一开始的 100 的时候模型效果最好，单纯提升它并不能提优模型。

2、如果训练轮数增加，生成效果是否一定更好？

我们将 noise_dim 设为 100，并保持所有配置不变，只改动训练轮数 epoch，得到如下表所示的训练结果：

Epochs	平均像素方差	样本间余弦相似度均值
10	0.5011	0.8862
20	0.2980	0.8417
50	0.2971	0.7777
100	0.3556	0.7185

表 2 不同 epochs 比较

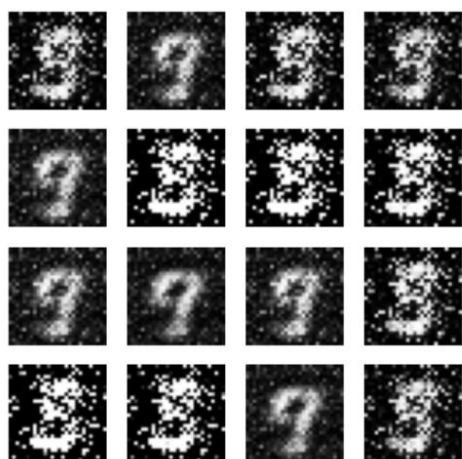


图 14 epochs=10

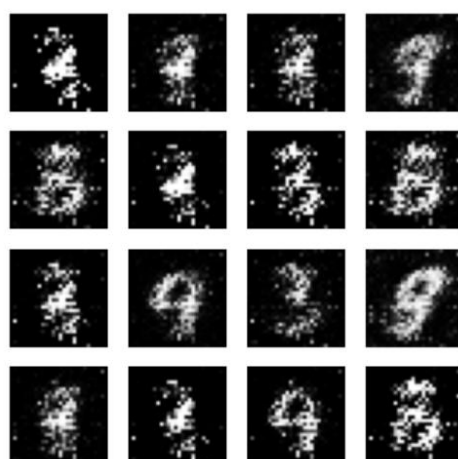


图 15 epochs=20

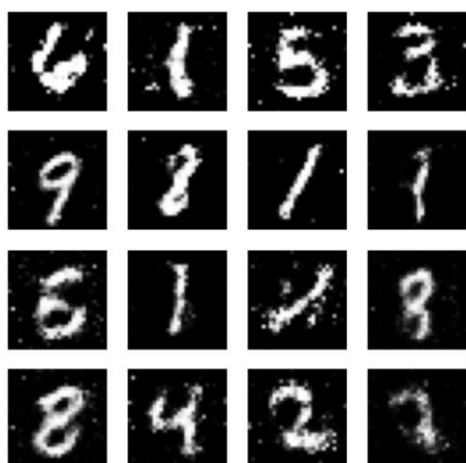


图 16 epochs=50

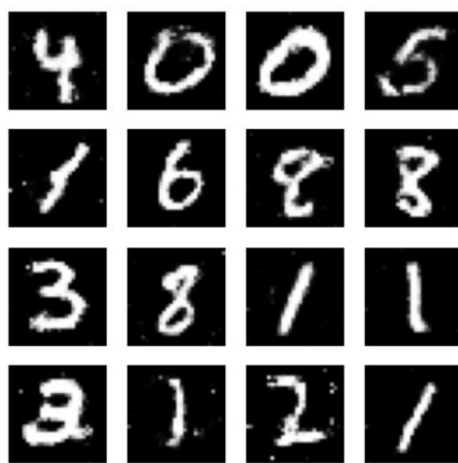


图 17 epochs=100

观察生成图片可以发现，随着 epochs 的增加，生成效果逐渐变好，手写数字的轮廓越来越清晰，已经逐步趋近于真实的 MNIST 数据集，另外观察表 2，发现样本间余弦相似度均值随着 epochs 增加而减小，说明生成图像的多样性在提升，但看平均像素方差这一指标，epochs=10 的时候最高，说明不同图像间的差异较大，分布广，但也有可能是因为一开始生成图像质量参差不齐而导致的，epochs=20 和 epochs=50 的平均像素方差差别不大，前者是因为训练不充分，后者则是因为训练逐渐趋于稳定，不同数字的差异开始体现，直到 epochs=100 的时候更为明显，训练趋于饱和，手写数字的质量已经很高了。

但就我做的这些实验以及结果来看，随着 epochs 的增加，训练效果是变好的，但估计再往上提，例如 200，500，提升应该就不大了，甚至可能由于过拟合而倒退。

3、对实验中的关键参数进行调整，观察并分析其对生成结果的影响，如学习率、网络结构、训练函数等，至少完成 2 种不同设置的对比实验，给出生成样本的可视化结果。

①关于学习率的研究讨论：

我们设定 epochs=20, noise_dim=100,并保持其他参数不变，仅修改学习率这一超参，由一开始的 lr=0.0002 作为对照组，设计了低学习率组和高学习率组，配置与相应实验指标如下表所示：

组别	学习率设置	平均像素方差	样本间余弦相似度均值
对照组（原参数）	lr = 0.0002	0.2857	0.7945
低学习率组	lr = 0.00005	0.4717	0.9994
高学习率组	lr = 0.001	0.4143	0.9981

表 3 不同学习率比较

为了更直观地观察不同学习率对训练的影响，我在训练过程中将每一个 epoch 的 loss 记录下来并绘制成曲线图，这样对比不同图像的曲线趋势（如震荡程度、收敛速度），可以更直观看到学习率对损失的影响，如图 21-23：

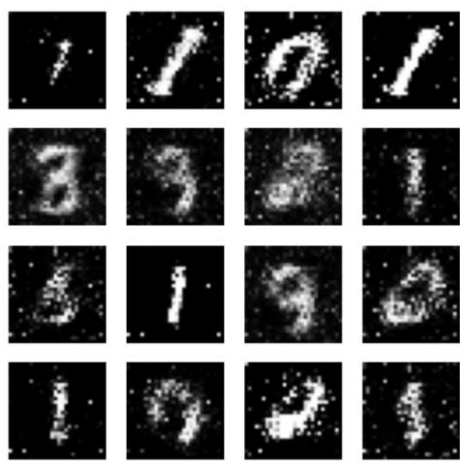


图 18 lr=0.0002



图 19 lr=0.00005

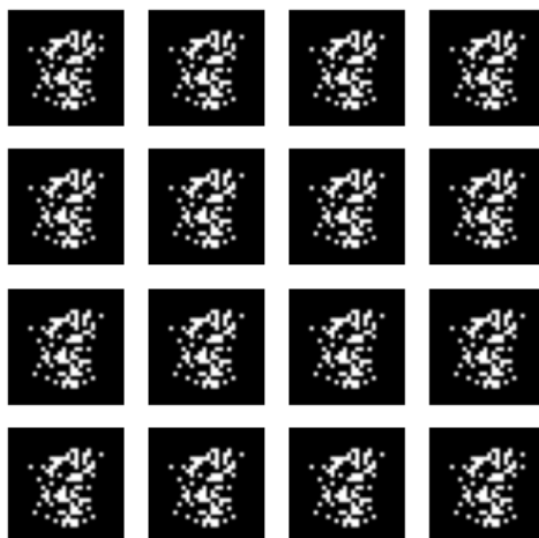


图 20 lr=0.001

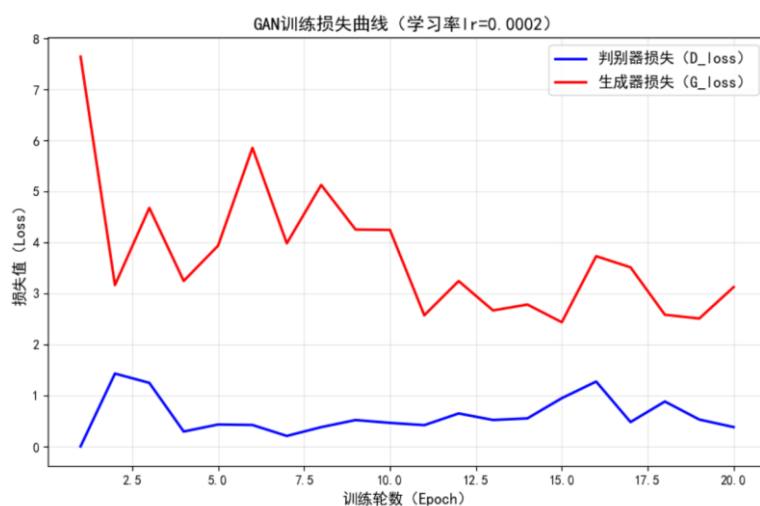


图 21 lr=0.0002 的 loss 曲线

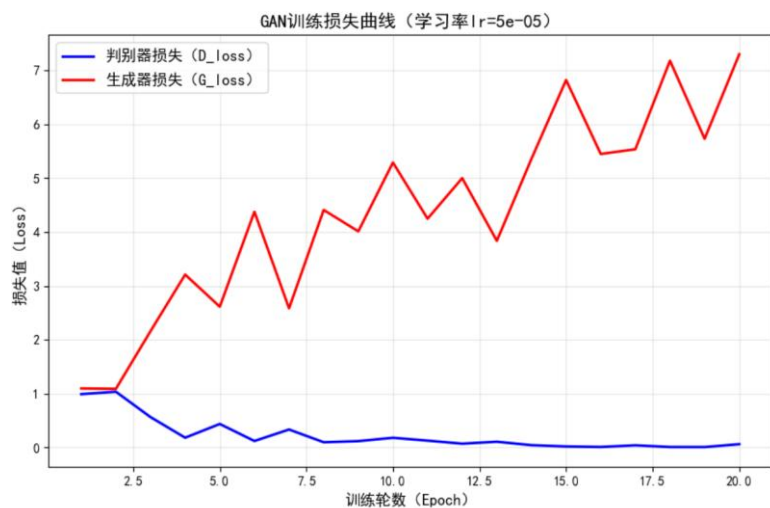


图 22 $lr=0.00005$ 的 loss 曲线

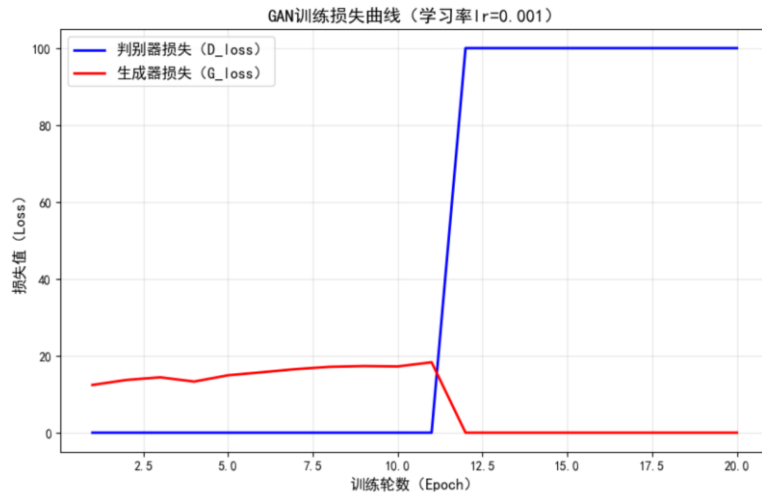


图 23 $lr=0.001$ 的 loss 曲线

观察损失图和生成图像以及表中指标，可以发现，对照组 ($lr=0.0002$) 呈现平衡的对抗状态，平均像素方差 (0.2857) 说明图像质量处于中间水平，说明生成图像有一定清晰度，但未达到最优；样本间余弦相似度 (0.7945)：是三组中最低的，说明生成样本的多样性相对最好，从实际肉眼看生成图像也是三者中相对最清晰的，loss 达到相对平衡的对抗状态，是三组中“质量与多样性兼顾”的最优选择；而低学习率组 ($lr=0.00005$) 中判别器压制生成器，多样性崩溃，判别器损失在训练初期就降至 0 附近并保持稳定，说明判别器快速占优，能轻松区分真假样本；生成器损失持续处于 3~7 的高区间，因学习率过低，生成器参数更新步长太小，无法跟上判别器的进化速度；而高学习率组 ($lr=0.001$) 中训练发散，对抗关系破裂，loss 发散而且更新不正常，生成器损失趋于 0，而判别器损失达到 100，训练崩溃了。

总结：GAN 的训练依赖“生成器与判别器的动态对抗平衡”，学习率是决定平衡状态的关键参数：

学习率过低：生成器更新过慢，被判别器压制，易出现“高质量但无多样性”的模式崩溃；

学习率过高：参数更新步长过大，训练易发散，对抗关系破裂；

学习率适中（如 0.0002）：能维持生成器与判别器的平衡，兼顾图像质量与多样性。

②关于网络结构的研究讨论：

为了更直观清晰的展示网络结构，我用 visio 画了网络结构的草图，如图 24 所示为我们一开始所使用的网络结构，生成器和判别器主体都为全连接层的堆叠：

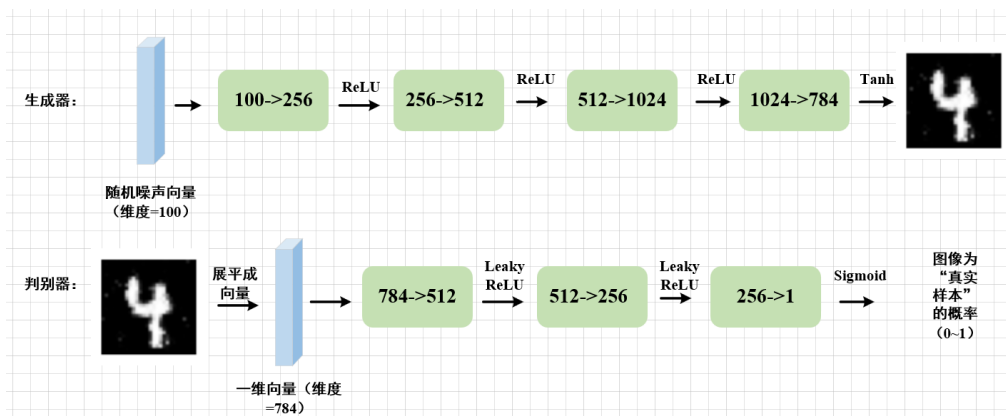


图 24 原始网络结构图

以“**调整隐藏层神经元数量**”为修改方向，我们做了两种不同的网络结构，网络 A 缩减了神经元数量，网络 B 扩增了神经元数量，具体结构分别如下图：

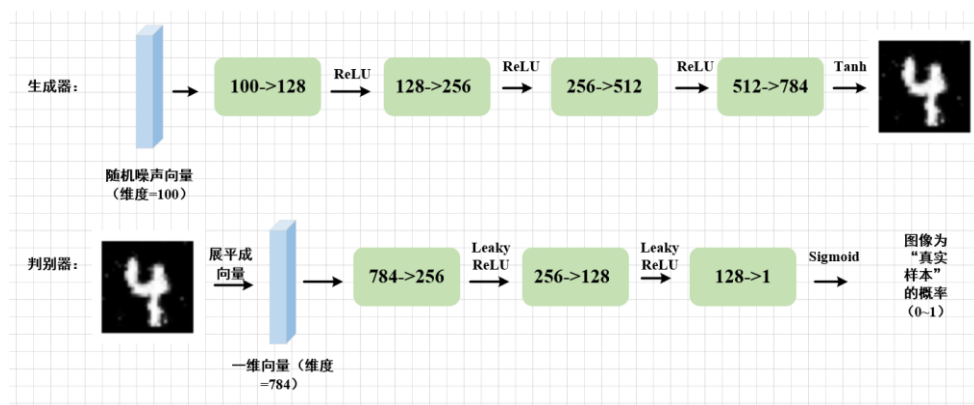


图 25 网络 A 结构图

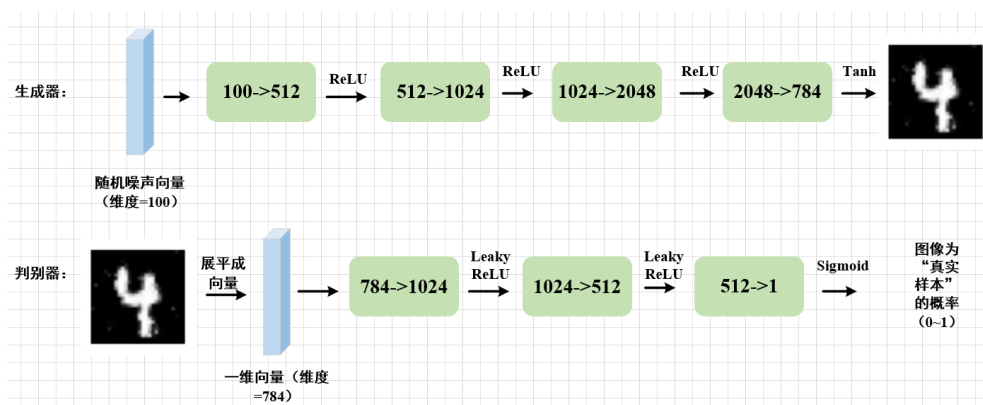


图 26 网络 B 结构图

实验设置为 epochs=20, noise_dim=100, 除网络结构外其余参数均不变, 得到如下表 and 图所示的实验结果:

网络结构	平均像素方差	样本间余弦相似度均值
原始网络	0.2551	0.8511
网络 A (缩减神经元)	0.2598	0.8637
网络 B (扩增神经元)	0.4289	0.9995

表 4 不同网络结构比较

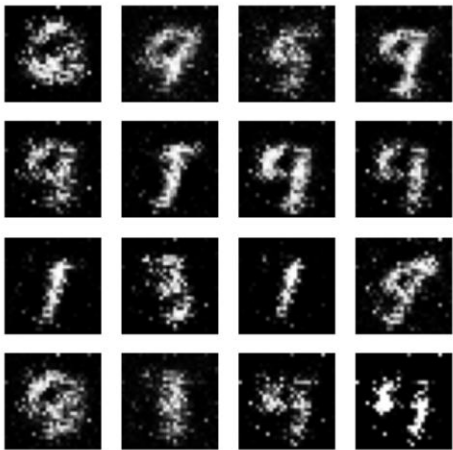


图 27 原始网络生成效果图

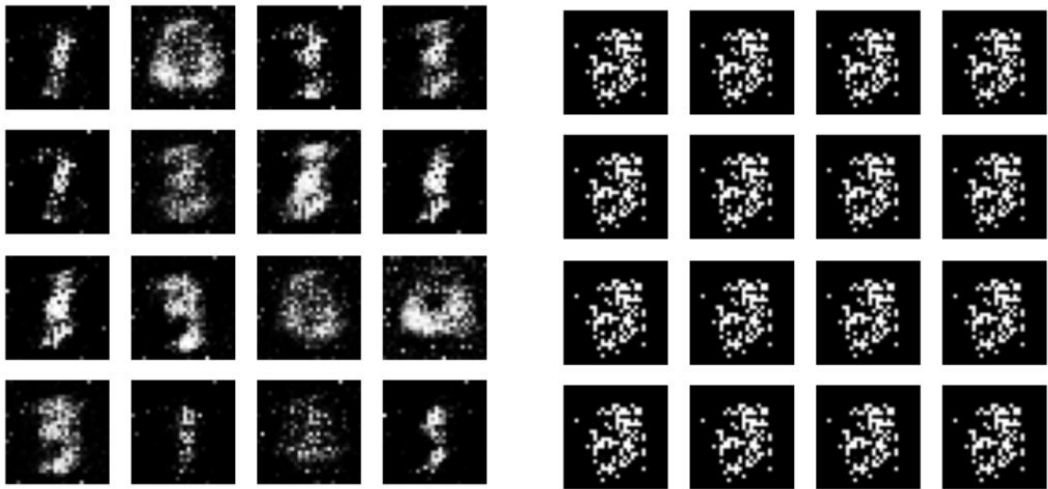


图 28 网络 A 生成效果图

图 29 网络 B 生成效果图

观察生成图像和指标可以发现，前两次也就是原始网络和网络 A 模型的效果其实差不多，指标比较相近，生成效果图的话原始网络要好一点，而网络 B 效果就差远了，在跑代码的时候也发现它的 loss 打印非常离谱，如图 30，应该是和学习率过高的情况一样训练崩溃了，生成器与判别器的对抗关系破裂。对比原始网络、缩减神经元的网络 A 与扩增神经元的网络 B：网络 A 的平均像素方差略高于原始网络、余弦相似度均值略升，说明缩减神经元后图像质量略有提升但多样性轻微下降，整体与原始网络差异不大；网络 B 的平均像素方差显著升高，但余弦相似度均值接近 1，生成样本的多样性严重崩溃。

总结：网络结构调整需平衡模型容量：缩减神经元对生成效果的影响有限，过度扩增神经元虽能提升图像质量，但易导致样本多样性丧失，需选择与任务适配的模型容量。

Epoch [1/20]	D_loss: 0.0010	G_loss: 7.1746
Epoch [2/20]	D_loss: 0.0000	G_loss: 24.9221
Epoch [3/20]	D_loss: 0.0000	G_loss: 10.2994
Epoch [4/20]	D_loss: 0.0000	G_loss: 10.8375
Epoch [5/20]	D_loss: 0.0000	G_loss: 50.7130
Epoch [6/20]	D_loss: 0.0000	G_loss: 49.8242
Epoch [7/20]	D_loss: 0.0000	G_loss: 49.1302
Epoch [8/20]	D_loss: 0.0000	G_loss: 48.4772
Epoch [9/20]	D_loss: 0.0000	G_loss: 47.8373
Epoch [10/20]	D_loss: 0.0000	G_loss: 47.2059
Epoch [11/20]	D_loss: 0.0000	G_loss: 46.6028
Epoch [12/20]	D_loss: 0.0000	G_loss: 45.9473
Epoch [13/20]	D_loss: 0.0000	G_loss: 45.2414
Epoch [14/20]	D_loss: 0.0000	G_loss: 42.9956
Epoch [15/20]	D_loss: 0.0000	G_loss: 55.4175
Epoch [16/20]	D_loss: 0.0000	G_loss: 55.1857
Epoch [17/20]	D_loss: 0.0000	G_loss: 55.0900
Epoch [18/20]	D_loss: 0.0000	G_loss: 54.8294
Epoch [19/20]	D_loss: 0.0000	G_loss: 54.0702
Epoch [20/20]	D_loss: 0.0000	G_loss: 53.9195

图 30 网络 B 训练 loss

另外，我还尝试在网络中加入 BN 层试试，生成器和判别器的全连接层后添加批量归一化层，得到如图 31 的效果图，相较于图 27，数字轮廓相对更明显一点，说明增加 BN 层有利于缓解梯度消失、提升训练稳定性，另外还获得了平均像素方差=0.6128，余弦相似度均值=0.7680 的高指标，两个指标均优于原始网络结构，不仅生成图像质量更好，多样性还更好。



图 31 增加 BN 层后的生成效果图

4、能否将全连接 GAN 改为卷积 GAN（DCGAN）？

我将生成器和判别器的网络结构重新设计了一遍，去掉了原有的全连接层，换上了卷积层，并且加入了 BN 层，代码与完整的网络结构图如下：

```
class Generator(nn.Module):
    def __init__(self, noise_dim=100, channels=1):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            # 输入: 噪声向量 (batch_size, noise_dim, 1, 1)
            nn.ConvTranspose2d(noise_dim, 128, kernel_size=7, stride=1, padding=0, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True), # 输出: (batch_size, 128, 7, 7)

            # 上采样到14x14
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True), # 输出: (batch_size, 64, 14, 14)

            # 上采样到28x28 (MNIST尺寸)
            nn.ConvTranspose2d(64, channels, kernel_size=4, stride=2, padding=1, bias=False),
            nn.Tanh() # 输出: (batch_size, 1, 28, 28), 像素值范围[-1,1]
        )

    def forward(self, z):
        # 将一维噪声向量reshape为4D张量: (batch_size, noise_dim, 1, 1)
        z = z.view(z.size(0), z.size(1), 1, 1)
        output = self.main(z)
        return output

G = Generator()
print(G)
```

图 32 生成器代码

```

class Discriminator(nn.Module):
    def __init__(self, channels=1):
        super(Discriminator, self).__init__()
        # 核心: 卷积层 (下采样) + BatchNorm + LeakyReLU
        self.main = nn.Sequential(
            # 输入: (batch_size, 1, 28, 28)
            nn.Conv2d(channels, 64, kernel_size=4, stride=2, padding=1, bias=False),
            nn.LeakyReLU(0.2, inplace=True), # 输出: (batch_size, 64, 14, 14)

            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True), # 输出: (batch_size, 128, 7, 7)

            nn.Conv2d(128, 1, kernel_size=7, stride=1, padding=0, bias=False),
            nn.Sigmoid() # 输出: (batch_size, 1, 1, 1), 对应真假概率
        )

    def forward(self, x):
        output = self.main(x)
        # 将输出展平为(batch_size, 1)
        return output.view(output.size(0), -1)

D = Discriminator()
print(D)

```

图 33 判别器代码

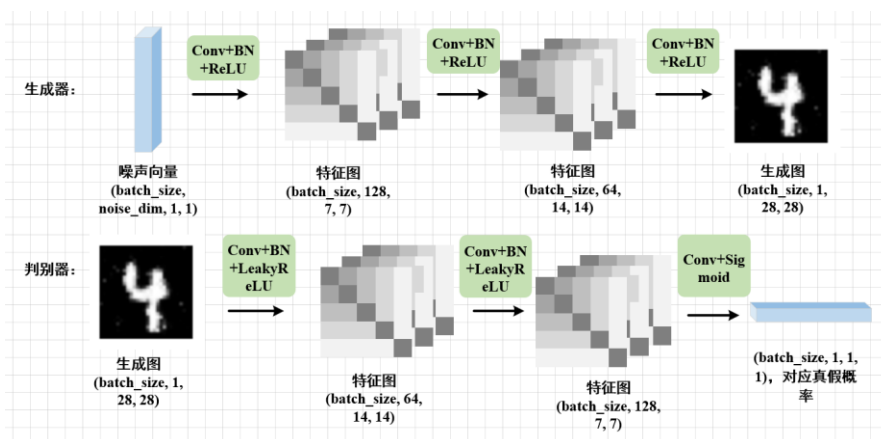


图 34 网络结构图

我的设计主要有以下四个关键调整点：

I.转置卷积的维度设计：从 7×7 逐步上采样到 14×14 再到 28×28 ，避免跨度过大导致的图像模糊，保证生成图像的细节；

II.通道数选择：生成器通道数从 $128 \rightarrow 64 \rightarrow 1$ ，判别器从 $1 \rightarrow 64 \rightarrow 128$ ，符合特征提取 / 生成的梯度流动规律；

III.优化器参数：DCGAN 经典配置 $\text{betas}=(0.5, 0.999)$ ，比默认的 $(0.9, 0.999)$ 更稳定，避免训练震荡；

IV.无偏置设计：卷积层设置 $\text{bias}=\text{False}$ ，因为批量归一化会抵消偏置的作用，减少参数数量提升效率。

训练结果如下：

平均像素方差	样本间余弦相似度均值
0.3629	0.7139

表 5 DCGAN 指标

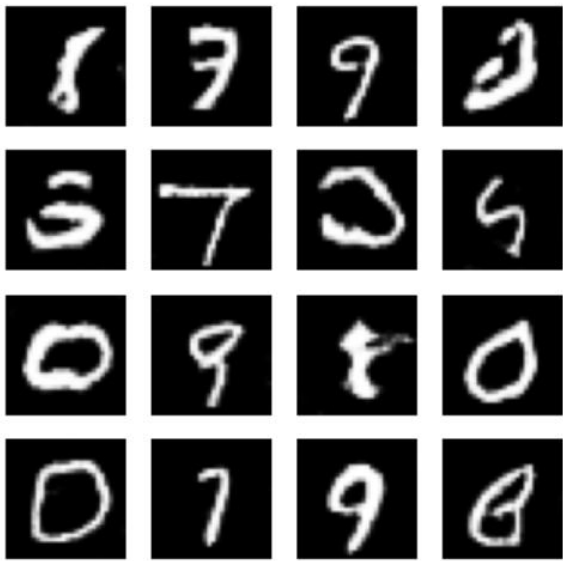


图 35 DCGAN 生成效果图

相比原有全连接 GAN，修改后的 DCGAN 有显著提升，同样 20 个 epoch 的训练，在图像质量上，生成的手写数字更清晰，笔画边缘更锐利，无模糊噪点；在多样性上，基本覆盖 0-9 所有数字，模式崩溃的概率大幅降低；而且从指标来看，平均像素方差和余弦相似度均值都要优于原本的模型，DCGAN 在 20 个 epoch 上取得的效果基本与原模型 100 个 epoch 训练的结果差不多，甚至还要小优，说明它拟合更快，效果更佳。

总结：

从基础全连接 GAN 的训练实验出发，我做了很多的实验与分析，依次探究噪声维度、训练轮数、学习率、网络结构对生成效果的影响，最终将全连接 GAN 改造为 DCGAN 的整个实验过程，让我对 GAN 的核心特性与调优逻辑形成了系统的认知：首先，GAN 的训练本质是维持生成器与判别器的动态对抗平衡，这一核心决定了各类参数调整并非“越大越好”或“越多越好”——训练轮数存在最优区间，超出后易出现过拟合或对抗失衡；噪声维度过高会增加生成

器的学习负担，导致图像质量与多样性双退化；学习率需适配对抗节奏，过高易引发训练发散，过低则会让生成器被判别器压制，唯有适中的学习率才能维持双方的平衡对抗。其次，网络结构的优化需兼顾模型容量与训练稳定性，增减全连接层神经元的实验表明模型容量需与 MNIST 手写数字生成任务匹配，而批量归一化层的加入能有效缓解梯度消失、提升训练平滑度；最后，将全连接层替换为卷积层的 DCGAN 改造，充分体现了卷积层对图像空间特征的捕捉优势，生成图像的清晰度、多样性实现了质的飞跃。

另外，由于本次实验中并没有一个很好的指标去量化模型的性能，所以我引入了两个指标，平均像素方差和样本间余弦相似度均值，前者用来评估生成图像质量，后者用来衡量生成图像多样性，让不同设置的实验模型有一个相对公平的比较方式，而不只是凭借肉眼观测生成图像的好坏，但是由于本实验中并未固定随机种子，每次跑出来的指标都是不一样的，实验具有随机性，但差别不大，在后续的拓展实验中可以在固定种子的情况下去做，这样更严谨，还可能继续把网络模型给它扩大下去，目前的参数量其实不高，在网络深度扩展的情况下提高 epochs，充分训练，相信能达到更好的训练效果。

三、 实验过程中遇到的问题和解决方案

1、gpu 的使用：

一开始我是用 cpu 训练的，我在我的环境中找不到 gpu，但后来发现是我的 base 环境安装的 PyTorch 版本不对，原先是 2.6.0+cpu，是纯 CPU 版本的 PyTorch，后来我又新建了一个环境，安装了 cuda 版本 12.1 的 PyTorch，现在可以识别并使用我电脑中的 gpu 了，训练速度比 cpu 要快。

2、张量设备不匹配：

我是用 gpu 来跑实验的，但是关于 numpy 的计算，需要转移到 cpu 上才能进行，在设计测试函数的时候，因为这一步报了很多张量设备不匹配的错误，说一些张量在 cpu 上，而另一些张量在 gpu 上，无法进行计算，所以在涉及 numpy 的计算时，要先转移到 cpu 上计算，然后计算完再转移出来，统一将生成器、判别器、训练数据、随机噪声移至同一设备（通过.to(device)）这样才不会报错。